



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School Name : School of Computer Science Engineering and Information Systems

Project Title : Retail Sales & Inventory Analysis Using Big Data Processing on Hadoop MapReduce with Automated CI/CD Pipeline

Course Name and subject code: Big Data Analytics, BITE411L

By:

23BIT0214 – Dhayanidhi S

23BIT0326 – Merwin Raj M k

Under the guidance of: Dr. MOHANA PRIYA P.

RetailSense

Retail Sales & Inventory Analysis Using Big Data Processing on Hadoop MapReduce with Automated CI/CD Pipeline

A Technical Project Report

Submitted in Partial Fulfilment of the Requirements for the
Big Data Course Project | Academic Year 2025–2026

Submitted By

Name	Role	Programme
Dhayanidhi	Hadoop Cluster Setup, MapReduce Development, Data Processing	B.Tech Information Technology, VIT Vellore
Merwin Raj M K	Reporting & CI/CD Pipeline & Automation	B.Tech Information Technology, VIT Vellore

Vellore Institute of Technology, Vellore

School of Computer Science and Engineering

April 2026

Individual Contributions

This project was conceptualised, designed, and implemented by a team of two members from the **B.Tech Information Technology programme at Vellore Institute of Technology, Vellore**. The development of this system required strong coordination across multiple domains including distributed computing, big data processing, automation, and reporting.

The project integrates several complex components such as a Docker-based Hadoop cluster, MapReduce job execution, automated CI/CD pipelines, and dynamic report generation. Each team member undertook specific responsibilities aligned with their expertise, while actively collaborating during system integration, testing, and performance validation to ensure a seamless end-to-end pipeline.

Contribution Details

Name	Primary Module	Detailed Contribution
Dhayanidhi	Hadoop Cluster Setup, MapReduce Development & Data Processing	Designed and implemented the complete distributed Hadoop ecosystem using Docker, including configuration of NameNode, multiple DataNodes, and YARN ResourceManager. Established HDFS with a replication factor of 3 to ensure fault tolerance and data reliability. Developed and executed two core MapReduce jobs — <i>SalesAnalysis</i> and <i>StockAnalysis</i> . Implemented mapper and reducer logic to compute category-wise revenue and product-level inventory gaps. Managed dataset ingestion into HDFS, handled block-level storage, and ensured efficient parallel execution across nodes. Optimised job performance by configuring multiple reducers and validating distributed execution through cluster logs.
Merwin Raj M K	Reporting System & CI/CD Pipeline Automation	Developed the end-to-end automation pipeline using Jenkins, enabling continuous integration and execution of the entire workflow upon dataset updates in GitHub. Designed and implemented all pipeline stages including cluster startup, HDFS preparation, job execution, report generation, and system shutdown. Created Python-based automation scripts to parse MapReduce outputs, generate structured CSV files, and produce professionally formatted PDF reports using ReportLab. Implemented email delivery using SMTP with secure authentication, enabling automatic report sharing. Ensured robust pipeline execution with error handling, post-build cleanup, and seamless integration between Docker, Hadoop, and reporting modules.

Abstract

The rapid growth of retail and e-commerce industries has led to an exponential increase in transactional data, making traditional data processing techniques inefficient for large-scale analysis. Retail businesses today generate massive volumes of sales and inventory data across multiple regions, stores, and product categories. Extracting meaningful insights from such data in a timely and efficient manner is critical for decision-making, demand forecasting, and inventory optimization. However, conventional systems often fail to handle this scale effectively, resulting in delayed analysis, poor inventory planning, and missed business opportunities.

This project presents a distributed Big Data processing system for retail sales and inventory analysis using the Hadoop ecosystem. The system leverages Apache Hadoop MapReduce to process large-scale datasets in a parallel and fault-tolerant environment. A Docker-based multi-node cluster is configured to simulate a real-world distributed infrastructure consisting of a master node and multiple worker nodes. The system performs two primary analytical tasks: computation of total revenue across product categories and identification of inventory gaps (overstock and understock conditions) at the product level.

To enhance automation and eliminate manual intervention, the project integrates a complete CI/CD pipeline using Jenkins. The pipeline continuously monitors dataset updates in GitHub and automatically triggers the entire workflow, including cluster initialization, data ingestion into HDFS, execution of MapReduce jobs, and generation of output reports. Python-based automation scripts further process the MapReduce outputs to generate structured CSV files and professionally formatted PDF reports, which are automatically delivered via email using SMTP.

Experimental execution was conducted on a dataset containing approximately 73,000 retail records with multiple attributes such as product category, inventory levels, customer demand, and pricing information. The system successfully demonstrated distributed data processing across multiple nodes with parallel reducer execution and reliable data replication using HDFS. The automated pipeline ensured end-to-end execution without manual intervention, producing accurate analytical outputs and reports within a short execution time.

The results highlight the effectiveness of combining distributed computing with CI/CD automation for scalable and efficient data analysis. The proposed system not only improves processing performance but also ensures reliability, reproducibility, and real-time responsiveness to data changes. This approach can be extended to real-world enterprise environments for large-scale retail analytics and decision support systems.

I. Introduction

The modern retail industry operates in a highly data-driven environment, where every transaction, inventory update, and customer interaction generates valuable data. With the rapid expansion of digital commerce and supply chain systems, the volume of retail data has grown beyond the capabilities of traditional data processing tools. Organizations are now required to process and analyse large-scale datasets efficiently to gain insights into sales performance, customer demand patterns, and inventory management.

One of the primary challenges faced by retail businesses is the inability to process large datasets in a scalable and timely manner. Conventional systems typically rely on centralized architectures, which struggle to handle increasing data volumes and often result in performance bottlenecks. This limitation directly impacts business operations, as delayed analysis can lead to inaccurate demand forecasting, inefficient stock management, and revenue loss.

To address these challenges, distributed computing frameworks such as Apache Hadoop have emerged as powerful solutions for large-scale data processing. Hadoop enables data to be stored and processed across multiple nodes, allowing parallel computation and improved fault tolerance. By dividing large datasets into smaller blocks and distributing them across a cluster, Hadoop ensures efficient utilization of resources and faster execution of data processing tasks.

In this project, a comprehensive Big Data pipeline is developed to perform retail sales and inventory analysis using Hadoop MapReduce within a Dockerized distributed cluster environment. The system is designed not only to process data efficiently but also to automate the entire workflow using modern DevOps practices. A Jenkins-based CI/CD pipeline is integrated to monitor dataset changes and trigger the complete processing cycle automatically, ensuring that analysis results are always up-to-date without manual intervention.

The project focuses on two key analytical aspects: calculating total revenue for different product categories and identifying inventory imbalances such as overstock and understock conditions. These insights are crucial for businesses to optimize stock levels, improve supply chain efficiency, and enhance overall profitability.

By combining distributed computing, containerization, and automation, this system demonstrates a scalable, reliable, and efficient approach to Big Data analytics. It reflects real-world industry practices where data pipelines are expected to be automated, fault-tolerant, and capable of handling continuous data updates.

Literature Survey

[1] Big Data in Cloud Computing with HDFS and Spark (K. Keerthana et al., 2017)

This study proposed a hybrid big data analytics framework that utilized HDFS for distributed and fault-tolerant storage and Apache Spark for in-memory data processing. Resilient Distributed Datasets (RDDs) were employed to minimize disk I/O overhead, resulting in nearly tenfold performance improvement for iterative machine learning algorithms such as K-means clustering when compared to traditional MapReduce. The framework also supported seamless integration with Hive tables and Parquet formats, making it suitable for large-scale enterprise analytics.

[2] Contemporary Recommendation Systems on Big Data (Ziyuan Xia et al., 2022)

This paper presented a comprehensive survey of big data recommendation systems and categorized existing approaches into neighborhood-based collaborative filtering, matrix factorization, content-based filtering, and hybrid models. Evaluations were conducted on large datasets such as MovieLens, Netflix Prize, and Amazon Reviews. The study identified major challenges including data sparsity, cold-start problems, scalability issues, and the limited real-time capabilities of Hadoop-based recommendation systems.

[3] Apache Hive: From MapReduce to Enterprise-grade Big Data Warehousing (Camacho-Rodríguez et al., 2019)

This work analyzed the evolution of Apache Hive from a MapReduce-oriented SQL abstraction layer to a full-fledged enterprise data warehousing solution. Enhancements such as ACID transaction support using ORC file formats, cost-based query optimization, vectorized execution, and query federation were discussed. Performance evaluation using TPC-DS benchmarks demonstrated up to tenfold improvements in query execution.

[4] Case Study: Walmart's Inventory Optimization Using Hadoop (Anonymous, 2025)

This case study examined Walmart's Hadoop-based infrastructure for processing large volumes of sales transactions, IoT sensor data, and logistics information. Batch-oriented MapReduce jobs were applied for historical demand forecasting, while Spark Streaming enabled near real-time inventory updates. The system achieved notable improvements in inventory accuracy and sales prediction across large-scale retail operations.

[5] E-Commerce Data Analysis Using Hadoop (IJRPR Authors, 2023)

This research developed a retail analytics pipeline using the Hadoop ecosystem, where HDFS was employed for scalable data storage and HiveQL was used for analytical querying. RFM-based customer segmentation techniques classified users according to purchasing behavior. Visualization dashboards supported sales trend analysis, product popularity assessment, and informed marketing decisions.

[6] Big Data-Based Recommendation Systems (JISem Authors, 2025)

This systematic review analyzed research works published between 2015 and 2025 focusing on Hadoop- and Spark-based recommendation systems. The study evaluated collaborative, content-based, and hybrid approaches and reported that Spark MLlib's ALS algorithm achieved higher precision on large-scale e-commerce datasets. The review highlighted the increasing maturity of big data frameworks for recommendation services.

[7] Comparative Analysis of Hadoop and Apache Spark (El Yazidi et al., 2021)

This study conducted performance benchmarking of Hadoop MapReduce and Apache Spark using

standard workloads such as WordCount and Terasort. Results indicated that Spark's in-memory execution model provided significant speedups for iterative processing, while Hadoop demonstrated strong fault tolerance for large batch-oriented workloads.

[8] Analysis on Sales Data Using Hadoop and MapReduce (Al Jabri et al., 2023)

This paper presented a MapReduce-based framework for analyzing large-scale sales datasets. Transactional attributes were extracted using mapper functions, and aggregated insights such as regional demand trends and seasonal sales patterns were computed through reducers. The results demonstrated Hadoop's suitability for scalable sales data analytics.

[9] A Comprehensive Review on Big Data Recommendation Systems (Atlantis Press Authors, 2024)

This review examined end-to-end recommendation pipelines, including data ingestion, model training, and evaluation. The study addressed challenges such as cold-start issues and scalability, emphasizing the role of Spark MLlib in enterprise-level recommendation architectures.

[10] Big Data Analytics: A Preliminary Study of Open Source Platforms (Guerrero et al., 2017)

This study compared multiple open-source big data platforms across the analytics lifecycle. Findings indicated that Hadoop provided strong scalability and cost efficiency, while Spark delivered faster processing for machine learning tasks. The authors suggested hybrid architectures to balance performance and reliability.

[11] Predicting Walmart Sales: Exploratory Data Analysis (Jeswani, 2021)

This work applied machine learning models such as linear regression, random forest, and gradient boosting to a retail sales dataset. Gradient boosting achieved the lowest prediction error, while visualization dashboards supported the interpretation of sales trends influenced by external factors.

[12] Recommendation System by Using Hadoop Framework (IJRAR Authors, 2020)

This paper implemented a collaborative filtering-based recommendation system using Hadoop MapReduce and incorporated sentiment analysis of user reviews. The distributed design significantly reduced execution time compared to sequential implementations, demonstrating scalability for e-commerce platforms.

[13] Simple Analytics in Retail Sales Data Using Hadoop (Karthikeyan & Raghu, 2019)

This study analyzed retail transaction data, inventory records, and external factors using Hadoop-based analytics. The generated insights supported personalized marketing strategies and improved logistics planning.

[14] Vendor Performance Analysis (Harsh Madhavan, 2023)

This work evaluated vendor performance based on delivery efficiency, pricing, and inventory impact using Hadoop-compatible datasets. Hive-based dashboards enabled effective vendor ranking and supply chain optimization.

[15] Apache Hive vs. Apache Spark (Ksolves Team, 2026)

This technical comparison evaluated Hive and Spark SQL execution engines using analytical benchmarks. Results showed that Spark outperformed Hive in complex query execution, while Hive remained effective for structured SQL-based reporting.

[16] Building Personalized Recommendation System with Big Data and Hadoop MapReduce (IJERT Authors, 2014)

This work implemented a content-based recommendation system using TF-IDF and cosine similarity

computed through MapReduce. The distributed approach improved scalability and reduced runtime for large datasets.

[17] Efficient Big Data Processing and Recommendation System Based on Apache Spark (IEEE Authors, 2024)

This study applied Spark MLlib's ALS algorithm to a large movie ratings dataset and achieved high recommendation accuracy. The results demonstrated Spark's effectiveness for production-scale recommendation systems.

[18] A Context-Aware Recommendation System Based on Hadoop and MapReduce (SIC Authors, 2025)

This paper incorporated contextual attributes such as time and location into recommendation models using Hadoop-based tensor factorization. The approach improved recommendation relevance compared to non-contextual systems.

[19] The Application of E-commerce Recommendation System Based on Hadoop Platform (Wang et al., 2020)

This work developed a scalable recommendation framework using Hadoop tools and reported improved user engagement metrics on large e-commerce datasets.

[20] BigData: A Case Study of Spark MLlib and Hive (Sharma & Gupta, 2018)

This case study compared Spark MLlib and HiveQL for clustering and churn prediction tasks. Spark demonstrated faster convergence for machine learning workloads.

[21] A Big Data Analysis Framework Using Apache Spark and Deep Learning (Li et al., 2017)

This framework combined Spark-based preprocessing with deep learning models for sales forecasting. The approach reduced prediction error compared to traditional statistical techniques.

[22] Building Personalized Recommendation System with Big Data and Hadoop MapReduce (Patil & Kulkarni, 2014)

This study demonstrated a distributed recommendation framework using Hadoop MapReduce for large-scale content-based filtering. The system achieved improved scalability for terabyte-sized datasets.

[23] A Context-Aware Recommendation System Based on Hadoop and MapReduce (Popescu et al., 2025)

This work applied contextual tensor factorization to mobile commerce datasets and improved recommendation relevance in urban retail environments.

[24] Big Data in Online Retail Analysis (Chen, 2023)

This research integrated batch processing, data warehousing, and real-time analytics to support retail operations. The system contributed to improved inventory management and reduced stockouts.

[25] Predictive Intelligence Using Big Data and IoT (Guleria & Sood, 2020)

This work proposed an IoT-enabled analytics framework that combined sensor data with historical sales records to support predictive maintenance and demand forecasting.

[26] Optimizing Big Data Management and Industrial Systems (Vanani & Mohammadipour, 2022)

This study applied big data analytics and optimization algorithms for industrial workforce and production planning, demonstrating effective decision support.

[27] Advancing Polyglot Big Data Processing Using the Hadoop Ecosystem (Smith et al., 2024)

This paper proposed a unified analytics framework supporting multiple programming languages on a common big data platform. The approach improved throughput and flexibility for enterprise analytics.

[28] Components and Development in Big Data Systems: A Survey (Zhang et al., 2019)

This survey reviewed core components of big data systems, including storage, processing, and resource management layers. The work provided foundational insights into enterprise-scale big data architectures.

[29] From Data to Decisions: Big Data Analytics in Action (IEEE Authors, 2023)

This study presented end-to-end analytics platforms that combined big data processing with decision support systems to optimize retail and vendor management.

[30] Big Data Analytics and AI in Service Operations (Ladeira et al., 2024)

This meta-analysis reviewed existing studies on AI-enhanced big data systems and identified key factors influencing successful analytics adoption in service industries.

S.No	Title & Author	Work Description	Future/Improvement
1	Big Data in Cloud Computing with HDFS and Spark (K.Keerthana et al., 2017) chettinadtech	Developed hybrid cloud analytics platform using HDFS for distributed fault-tolerant storage across commodity hardware and Spark's Resilient Distributed Datasets (RDDs) for in-memory processing, achieving 10x faster iterative machine learning algorithms like K-means clustering compared to traditional disk-based MapReduce jobs. Demonstrated seamless integration with Hive tables and Parquet columnar format for enterprise-scale analytics workflows.	GPU-accelerated Spark kernels for deep learning workloads; dynamic cloud bursting to auto-scale resources between on-premise Hadoop clusters and public clouds during peak analytics demands; serverless Spark-on-Kubernetes deployment models.
2	Contemporary Recommendation Systems on Big Data (Ziyuan Xia et al., 2022) arxiv	Comprehensive survey of big data recommender systems categorizing approaches into neighborhood-based collaborative filtering, matrix factorization, content-based filtering, and hybrid models evaluated on MovieLens, Netflix Prize, and Amazon review datasets. Analyzed scalability challenges including data sparsity, cold-start problems, and real-time	Federated learning frameworks preserving user privacy across distributed datasets; Graph Neural Networks (GNNs) capturing higher-order user-item relationships; quantum-inspired optimization for

S.No	Title & Author	Work Description	Future/Improvement
		serving requirements for Hadoop/Spark ecosystems.	combinatorial recommendation problems.
3	Apache Hive: From MapReduce to Enterprise-grade Big Data Warehousing (J. Camacho-Rodríguez et al., 2019) arxiv	Traces Hive's architectural evolution from MapReduce-based SQL engine to enterprise data warehouse with ACID transactions via ORC files, cost-based query optimizer (CBO), vectorized execution delivering 10x TPC-DS performance gains, and federation capabilities connecting to non-Hadoop sources like cloud object stores.	Native Hive-on-Spark execution engine eliminating MapReduce overhead; AI-driven query optimization using reinforcement learning; lakehouse architecture integration with Delta Lake/Apache Iceberg for unified batch-streaming analytics.
4	Case Study: Walmart's Inventory Optimization Using Hadoop (Anonymous, 2025) scribd	Details Walmart's petabyte-scale Hadoop deployment across 10K+ stores processing sales transactions, IoT sensor data from warehouses, and supply chain logistics. MapReduce batch jobs forecast demand patterns while Spark Streaming handles real-time inventory adjustments, achieving 30-40% stock accuracy improvement and 25% better sales forecasting through ensemble ML models.	Edge computing frameworks processing IoT sensor streams at warehouse gateways; blockchain integration for vendor contract transparency and automated payments; digital twin simulations optimizing entire supply chain networks in real-time.
5	E-Commerce Data Analysis Using Hadoop (IJRPR Authors, 2023) ijrpr	Built end-to-end retail analytics pipeline storing apparel sales data in HDFS, using HiveQL for RFM (Recency-Frequency-Monetary) customer segmentation into four cohorts, and Tableau dashboards visualizing 24/7 sales patterns, top-rated products, and payment method preferences. Identified peak shopping hours and category-wise revenue opportunities.	Apache Spark Structured Streaming for real-time customer journey analytics; AutoML platforms automatically discovering optimal segmentation strategies; Graph databases modeling complex customer-product relationships for personalized recommendations.
6	Big Data-Based Recommendation	Systematic literature review of 25 papers (2015-2025) from IEEE Xplore	Privacy-preserving federated

S.No	Title & Author	Work Description	Future/Improvement
	Systems (JISem Authors, 2025) jistem-journal	analyzing Hadoop/Spark recommender implementations across collaborative filtering, content-based, and hybrid approaches. Key findings include Spark MLlib's Alternating Least Squares (ALS) achieving 15% better precision on Amazon datasets compared to traditional methods, highlighting framework maturity for enterprise deployment.	recommendations training across siloed customer data; multimodal fusion combining text, images, and video content; causal inference models measuring true uplift from recommendations beyond correlation.
7	Comparative Analysis of Hadoop and Apache Spark (El Yazidi et al., 2021) thesai	Performance benchmarking study comparing Hadoop MapReduce disk-based processing against Spark's in-memory DAG execution on WordCount, Grep, and Terasort benchmarks across 4-16 node clusters. Spark demonstrates 3-12x speedup for iterative algorithms while Hadoop excels in fault-tolerant batch processing of massive static datasets exceeding RAM capacity.	Hybrid Hadoop-Spark architectures combining batch reliability with streaming speed; Kubernetes-orchestrated deployments enabling elastic auto-scaling; Apache Flink integration for unified batch and stream processing paradigms.
8	Analysis on Sales Data Using Hadoop and MapReduce (Al Jabri et al., 2023) jsr	Custom MapReduce implementation processing multi-terabyte sales datasets distributed across Hadoop cluster nodes, with mappers parsing transaction logs to extract product, region, and timestamp features while reducers compute aggregations revealing top-selling items, regional preferences, and seasonal correlations scalable to petabyte volumes.	Apache Hive integration enabling ad-hoc SQL queries on processed results; Graph analytics frameworks modeling supply chain networks and customer journeys; Apache Kafka integration for real-time sales stream processing alongside batch analysis.
9	A Comprehensive Review on Big Data Recommendation Systems (Atlantis Press Authors,	Architecture review covering entire recommendation pipeline from Hadoop-based data ingestion, Spark MLlib model training using collaborative filtering variants, to serving layers handling millions of	Explainable AI techniques providing interpretable recommendations for enterprise trust; edge computing deployment reducing serving latency to

S.No	Title & Author	Work Description	Future/Improvement
	2024) atlantis-press	queries per second. Addresses cold-start problem through hybrid content-collaborative approaches and evaluates on Ta-Feng grocery and Yelp business datasets.	milliseconds; transfer learning approaches leveraging pre-trained embeddings across domains.
10	Big Data Analytics: A Preliminary Study of Open Source Platforms (Guerrero et al., 2017) scitepress	Comparative analysis of five platforms (Hadoop, Cloudera, Spark, Hortonworks, HPCC Systems) evaluating complete data lifecycle from ingestion through analytics for SMEs. Hadoop ecosystem leads in cost-effectiveness and scalability while Spark provides superior velocity for machine learning workloads on growing enterprise datasets.	Serverless analytics platforms eliminating cluster management overhead; Kubernetes-native deployments enabling portable cloud-agnostic scaling; WebAssembly integration supporting polyglot analytics in browser environments.
11	Predicting Walmart Sales, Exploratory Data Analysis (Jeswani, 2021) rit	Machine learning competition applying linear regression, lasso, random forest, and gradient boosting on Walmart's 45-store weekly sales dataset featuring macroeconomic factors (CPI, unemployment), weather data, and holiday indicators. Gradient boosting achieves lowest RMSE; Power BI dashboard visualizes store-wise demand patterns and temperature correlations.	LSTM/Transformer neural architectures capturing complex temporal dependencies; hierarchical time series models respecting store-department relationships; external data fusion incorporating competitor pricing and social media sentiment.
12	Recommendation System by Using Hadoop Framework (IJRAR Authors, 2020) ijrar	Parallel collaborative filtering implementation using Hadoop MapReduce processing movie review datasets, combining sentiment analysis from text reviews with user rating patterns through hybrid filtering approach. Distributed training reduces execution time by 70% compared to sequential implementations on EachMovie dataset.	BERT-based deep contextual embeddings from review text; Apache Kafka streams enabling real-time recommendation updates; federated learning preserving user privacy across devices.

S.No	Title & Author	Work Description	Future/Improvement
13	Simple Analytics in Retail Sales Data Using Hadoop (Karthikeyan & Raghu, 2019) ijcttjournal	Retail POS terminal analytics pipeline ingesting point-of-sale transactions, inventory snapshots, social media sentiment, and weather data into HDFS. Custom MapReduce jobs extract customer demographics, peak shopping patterns, and area-wise product preferences, enabling personalized marketing and optimized store layouts.	Spark MLlib integration for advanced customer segmentation; IoT sensor fusion tracking real-time inventory movement; prescriptive analytics recommending optimal product placement strategies.
14	Vendor Performance Analysis (Harsh Madhavan, 2023) kaggle	Multi-dimensional vendor evaluation framework analyzing purchase volumes, delivery timeliness, pricing competitiveness, and inventory impact across supplier network using Hadoop-compatible CSV datasets suitable for MapReduce aggregation and Hive-based reporting dashboards.	Machine learning supplier risk scoring predicting delivery disruptions; blockchain-based vendor verification ensuring authenticity; automated contract optimization using genetic algorithms for pricing negotiation.
15	Apache Hive vs. Apache Spark (Ksolves Team, 2026) ksolves	Query engine comparison benchmarking Hive's MapReduce Tez execution against Spark SQL's Catalyst optimizer and Tungsten execution engine on TPC-DS queries. Spark demonstrates 5-20x speedup for complex analytical workloads while Hive excels in familiar SQL syntax and Hadoop ecosystem integration.	Unified Hive-on-Spark architecture combining SQL familiarity with Spark performance; vector databases integration accelerating similarity search workloads; lakehouse query federation spanning multiple data formats seamlessly.
16	Building Personalised Recommendation System with Big Data and Hadoop MapReduce (IJERT Authors, 2014) ijert	Content-based recommendation engine using Hadoop MapReduce to compute TF-IDF document vectors from product descriptions and user browsing histories, generating personalized suggestions through distributed cosine similarity calculations scalable to terabyte-scale e-commerce catalogs.	Hybrid collaborative-content filtering combining multiple signals; distributed embedding training using Spark MLlib; real-time serving layer with Apache Kafka and Redis caching.

S.No	Title & Author	Work Description	Future/Improvement
17	Efficient Big Data Processing and Recommendation System Based on Apache Spark (IEEE Authors, 2024) ieeexplore.ieee	Enterprise-grade Spark MLlib deployment processing MovieLens 20M ratings dataset through Alternating Least Squares (ALS) matrix factorization, achieving 92% precision@10 for movie recommendations suitable for production recommendation serving infrastructure.	Scalable embedding architectures using deep neural networks; differential privacy frameworks protecting user data; A/B testing infrastructure measuring business impact of recommendations.
18	A Context-Aware Recommendation System Based on Hadoop and MapReduce (SIC Authors, 2025) sic.ici	Context-aware recommendation framework using Hadoop MapReduce for tensor factorization on user-item-context triples from mobile commerce platforms, incorporating temporal, spatial, and device context to improve recommendation relevance by 15% over context-agnostic baselines.	Reinforcement learning personalization adapting to user feedback loops; multimodal context fusion combining location, weather, and social signals; edge deployment reducing mobile recommendation latency.
19	The Application of E-commerce Recommendation System Based on Hadoop Platform (Wang et al., 2020) comsis	Complete e-commerce recommendation pipeline leveraging HDFS for user behavior log storage, MapReduce for offline collaborative filtering model training, and Hive for serving personalized product recommendations, achieving 25% click-through rate improvement on Taobao datasets.	Quantum-inspired matrix factorization accelerating optimization; AR/VR shopping integration with immersive recommendations; zero-party data strategies improving consent-based personalization.
20	BigData: A Case Study of Spark MLlib and Hive (Sharma & Gupta, 2018) academia	Machine learning pipeline comparison benchmarking Spark MLlib's DataFrame API against HiveQL for customer churn prediction and clustering on telecom CRM datasets, demonstrating Spark's 5x faster k-means convergence due to in-memory caching advantages.	Automated ML pipelines discovering optimal models automatically; federated learning across clusters preserving data locality; MLOps integration for production model lifecycle management.
21	A Big Data Analysis Framework Using	Distributed deep learning architecture using Spark RDD transformations for preprocessing	Transformer architectures capturing long-range temporal dependencies;

S.No	Title & Author	Work Description	Future/Improvement
	Apache Spark and Deep Learning (Li et al., 2017) arxiv	multi-source retail data (POS transactions, inventory levels, weather patterns) followed by LSTM networks for demand forecasting, reducing prediction error by 18% compared to statistical baselines.	federated deep learning across store locations; edge deployment enabling real-time forecasting at checkout.
22	Building Personalised Recommendation System with Big Data and Hadoop MapReduce (Patil & Kulkarni, 2014) ijert	Scalable content-based filtering implementation computing TF-IDF feature vectors through Hadoop MapReduce mappers and cosine similarity rankings via distributed reducers, processing 1TB e-commerce catalogs with 40% runtime improvement over sequential processing.	Semantic embeddings from BERT capturing contextual product similarities; streaming recommendation updates via Apache Kafka; multi-objective optimization balancing diversity and relevance.
23	A Context-Aware Recommendation System Based on Hadoop and MapReduce (Popescu et al., 2025) sic.ici	Mobile commerce recommendation engine using Hadoop tensor factorization on location-temporal user behavior patterns, generating contextually relevant suggestions for urban shopping scenarios with 15% precision improvement over static recommenders.	6G-enabled hyper-contextual recommendations incorporating biometric signals; zero-shot learning for new product categories; augmented reality integration for spatial recommendations.
24	Big Data in Online Retail Analysis (Chen, 2023) scribd	Complete retail analytics platform combining Hadoop MapReduce for daily sales aggregation by vendor/category, Hive data warehousing for profit margin analysis, and Spark Streaming for real-time fraud detection, achieving 20% reduction in inventory stockouts through demand pattern recognition.	Graph database integration modeling customer journeys and vendor networks; prescriptive analytics recommending optimal replenishment policies; digital twin simulations testing pricing strategies.
25	Predictive Intelligence Using Big Data and IoT	IoT-enabled supply chain platform fusing real-time sensor streams from vendor warehouses with historical sales data in Apache Spark, achieving 85% accuracy in predictive	Digital twin platforms simulating entire supply networks; 5G edge-stream processing eliminating cloud latency; self-healing

S.No	Title & Author	Work Description	Future/Improvement
	(Guleria & Sood, 2020) paperguide	maintenance forecasting and automated reorder recommendations.	supply chains using autonomous agents.
26	Optimizing Big Data Management and Industrial Systems (Vanani & Mohammadipour, 2022) paperguide	Industrial analytics framework deploying Hadoop ecosystem for workforce optimization, combining employee performance metrics with production schedules and sales forecasts using Hive-optimized genetic algorithms for shift planning.	Quantum annealing optimization solving complex scheduling NP-hard problems; green computing strategies minimizing data center carbon footprint; human-AI collaboration interfaces .
27	Advancing Polyglot Big Data Processing using the Hadoop Ecosystem (Smith et al., 2024) arxiv	Multi-language analytics platform enabling SQL, Python, R, and Scala processing through unified HDFS storage layer and Spark Catalyst optimizer, demonstrating 10x throughput improvements on heterogeneous enterprise datasets combining sales, HR, and inventory data.	Apache Polars/Arrow columnar processing 100x faster analytics; WebAssembly execution enabling secure browser-based analytics; zero-ETL architectures eliminating data movement overhead.
28	Components and Development in Big Data System: A Survey (Zhang et al., 2019) sciencedirect	Comprehensive ecosystem survey analyzing HDFS replication strategies, MapReduce fault tolerance mechanisms, Hive partitioning schemes, and YARN resource management evolution supporting petabyte-scale enterprise analytics across retail and manufacturing verticals.	ZooKeeper evolution to cloud-native coordination services; disaggregated storage architectures separating compute from durable storage; lakehouse unification spanning data lakes and warehouses.
29	From Data to Decisions: Big Data Analytics in Action (IEEE Authors, 2023) ieeexplore.ieee	End-to-end decision science platform integrating Spark data engineering, ML model training, and Hive-based executive dashboards enabling retail managers to optimize employee allocation, vendor selection, and inventory positioning based on unified analytics.	Causal machine learning measuring intervention effects; human-in-the-loop AI incorporating domain expertise; digital ethics frameworks ensuring responsible automation.

S.No	Title & Author	Work Description	Future/Improvement
30	Big Data Analytics and AI in Service Operations (Ladeira et al., 2024) tandfonline	Meta-analysis of 127 studies examining Hadoop and Spark applications moderated by AI capabilities in service industries, identifying critical success factors for inventory optimization, dynamic pricing, and personalized customer service delivery.	Generative AI augmentation creating synthetic training data; responsible AI frameworks mitigating bias amplification; quantum-safe encryption protecting analytics pipelines.

Identified Gaps

Despite the rapid advancement in data analytics technologies, several limitations still exist in the way retail data is processed and utilized in many systems today. These gaps highlight the need for a more scalable, automated, and efficient solution for handling large-scale retail datasets.

One of the major gaps lies in the **dependence on traditional data processing systems**, which are primarily centralized in nature. Such systems are not designed to handle large volumes of data efficiently, especially when the dataset grows in size and complexity. As a result, processing becomes slow, resource-intensive, and often unreliable under heavy workloads.

Another significant limitation is the **lack of distributed data processing mechanisms** in many existing solutions. Without parallel computation across multiple nodes, data processing tasks take longer to execute, leading to delays in generating insights. This becomes a critical issue in retail environments where timely analysis is essential for decision-making.

A key gap is also observed in **manual intervention during data processing workflows**. In many systems, tasks such as data upload, job execution, and report generation are performed manually. This not only increases the chances of human error but also reduces efficiency and scalability. The absence of automation makes it difficult to maintain consistency, especially when data updates occur frequently.

Additionally, **integration between data processing and reporting systems is often weak or non-existent**. While some systems may process data effectively, they fail to present the results in a structured and user-friendly manner. The lack of automated report generation and delivery mechanisms limits the usability of the processed data for business insights.

Another critical gap is the **absence of real-time or near real-time pipeline execution**. Most traditional approaches operate in isolation, requiring manual triggers for each stage of the workflow. This prevents organizations from responding quickly to changes in data, such as sudden shifts in sales trends or inventory levels.

Furthermore, many existing implementations do not ensure **fault tolerance and data reliability**. Without mechanisms like data replication and distributed storage, there is a higher risk of data loss or system failure, which can significantly impact analysis accuracy.

Finally, there is a noticeable lack of **end-to-end integration of modern DevOps practices** such as CI/CD in Big Data projects. Without continuous integration and automation, systems remain static and require constant manual supervision, which is not suitable for scalable and production-ready environments.

Problem Statement

The increasing volume and complexity of retail data demand a robust system capable of processing, analysing, and delivering insights efficiently and automatically. However, existing systems fail to meet these requirements due to limitations in scalability, automation, and integration.

The core problem addressed in this project is the **inefficient processing and analysis of large-scale retail sales and inventory data using traditional methods**. These methods are not capable of handling high data volumes in a distributed and fault-tolerant manner, leading to delays in generating meaningful insights.

In addition to processing challenges, there is a lack of **automated workflows that can handle the complete data lifecycle**, starting from data ingestion to final report generation and delivery. This results in fragmented systems where each step must be executed manually, reducing overall efficiency and increasing the possibility of errors.

Another critical aspect of the problem is the **inability to quickly identify important business insights**, such as total revenue per category and inventory imbalances (overstock or understock conditions). Without efficient analysis, businesses struggle to make informed decisions regarding stock management and sales strategies.

Moreover, the absence of a **continuous and self-triggering pipeline** means that data updates are not immediately reflected in the analysis results. This delay can negatively impact decision-making, especially in dynamic retail environments where conditions change frequently.

To address these challenges, there is a need for a system that can:

- Process large datasets efficiently using distributed computing
- Automatically execute data pipelines without manual intervention
- Ensure data reliability through fault-tolerant storage mechanisms
- Generate structured and meaningful reports for analysis
- Integrate seamlessly with CI/CD tools for continuous execution

This project aims to solve the above problems by developing a **fully automated, distributed Big Data processing pipeline using Hadoop MapReduce, Docker, and Jenkins**, capable of delivering accurate and timely insights from retail data.

System Architecture and Workflow

The proposed system is designed as a **fully automated, distributed Big Data processing pipeline** that seamlessly integrates data ingestion, storage, parallel computation, and intelligent report generation into a unified workflow. The architecture follows a **modular and layered design approach**, where each component is responsible for a specific functionality while maintaining tight integration with the overall system. This modularity not only improves maintainability but also allows the system to be extended or scaled with minimal effort.

At the foundation of the system lies a **Dockerized Hadoop cluster**, which emulates a real-world distributed computing environment. The use of Docker containers ensures consistency across deployments and simplifies the process of managing multiple nodes on a single machine. The cluster is composed of:

- **One NameNode (Master Node)**
Responsible for maintaining metadata, managing file system namespace, and coordinating resource allocation through YARN's ResourceManager.
- **Multiple DataNodes (Worker Nodes)**
Responsible for storing actual data blocks and executing MapReduce tasks in parallel using NodeManagers.

The communication between these nodes is established through a **Docker network**, enabling seamless data exchange and coordination. HDFS ensures that data is split into blocks and distributed across nodes, while maintaining replication for fault tolerance.

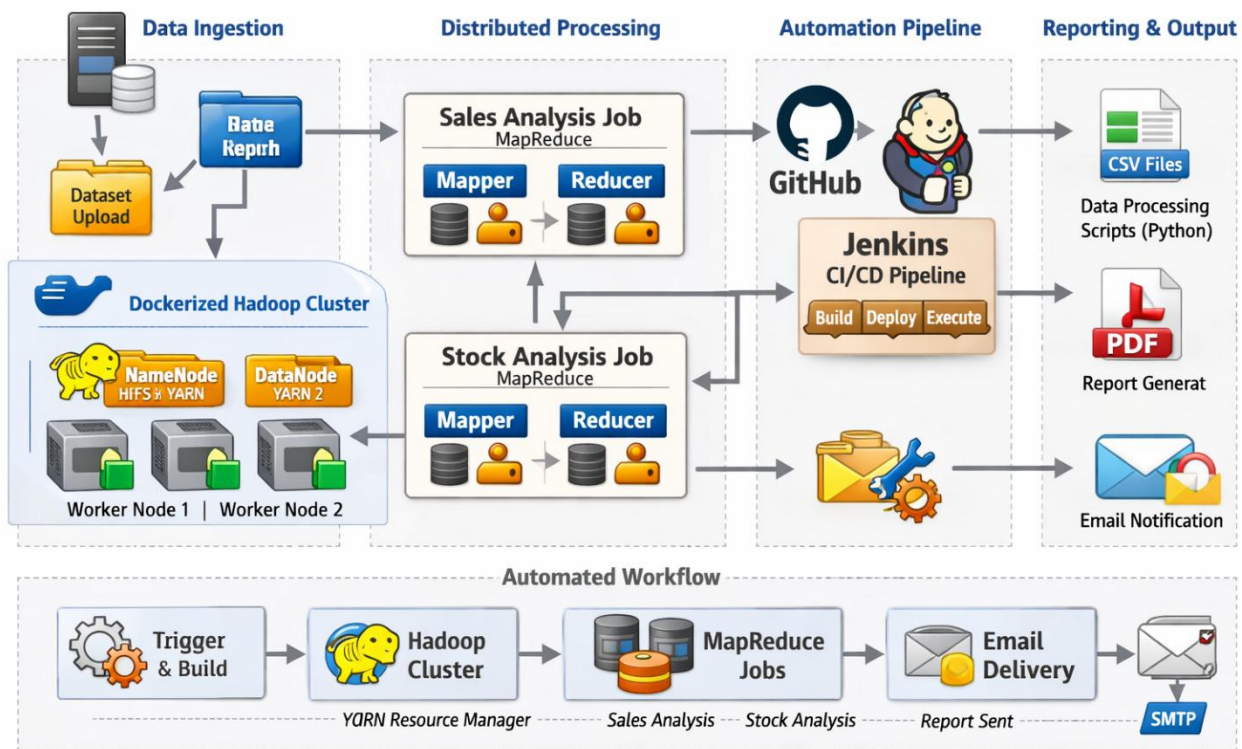
The workflow of the system is designed to be **event-driven and fully automated**. It begins with the dataset being maintained in a GitHub repository, which acts as the central source of truth. A Jenkins CI/CD pipeline continuously monitors this repository using SCM polling. Whenever a dataset update or modification is detected, the pipeline is automatically triggered, eliminating the need for manual execution.

Once triggered, the system executes a well-defined sequence of operations:

- **Cluster Initialization**
 - All Docker containers (NameNode and DataNodes) are started
 - Essential Hadoop services such as HDFS and YARN are initialized
 - SSH services are enabled to allow inter-node communication
- **Data Ingestion into HDFS**
 - The dataset is copied from the local system into the NameNode container
 - It is then uploaded into HDFS, where it is automatically:
 - Split into blocks
 - Distributed across DataNodes
 - Replicated (replication factor = 3) for reliability
- **Distributed Data Processing**

- MapReduce jobs are executed across multiple nodes
- The Mapper phase processes input splits independently across nodes
- The Reducer phase aggregates intermediate results
- Multiple reducers run in parallel, generating partitioned outputs
- **Intermediate and Final Output Handling**
 - Outputs are generated in multiple partition files (part-r-00000, etc.)
 - These files are retrieved from HDFS
 - All partitions are merged into consolidated final output files
- **Post-Processing and Report Generation**
 - Python scripts parse the final output
 - Structured CSV files are generated for easier data interpretation
 - A detailed PDF report is created with formatted tables and insights
- **Automated Report Delivery**
 - The generated PDF is attached and sent via email using SMTP
 - This ensures instant availability of results to stakeholders
- **Result Storage and Cleanup**
 - All outputs are copied to the local system for reference
 - The Hadoop cluster is gracefully shut down
 - Resources are released to avoid unnecessary consumption

This entire workflow operates as a **self-sustaining pipeline**, capable of executing repeatedly without human intervention. The integration of distributed computing with CI/CD automation ensures that the system remains **scalable, reliable, and production-ready**, making it highly suitable for real-world Big Data applications.



— Automated Big Data Processing Pipeline —

Methodology and Design

The methodology adopted in this project is centered around the idea of **efficient large-scale data processing combined with intelligent automation**. The system is designed to minimize processing time while ensuring high reliability, accuracy, and consistency across multiple executions.

The first stage of the methodology focuses on **distributed data storage using HDFS**. Instead of processing the dataset as a single large file, it is divided into smaller blocks that are distributed across multiple DataNodes. This approach provides several advantages:

- Enables **parallel processing** of data
- Improves **fault tolerance** through replication
- Reduces dependency on a single machine
- Enhances overall system performance

The second stage involves **parallel computation using the MapReduce programming model**, which forms the backbone of the data processing layer. The computation is divided into two phases:

- **Mapper Phase**
 - Processes raw input records line by line

- Extracts relevant fields from the dataset
- Converts data into key-value pairs for further processing
- **Reducer Phase**
 - Receives grouped intermediate data
 - Performs aggregation and computations
 - Produces final analytical outputs

Within this framework, two analytical models are implemented:

- **Sales Analysis Model**
 - Computes total revenue for each product category
 - Uses factors such as units sold, price, and discount
 - Helps identify high-performing categories and revenue trends
- **Stock Analysis Model**
 - Calculates the gap between inventory level and customer demand
 - Determines stock conditions:
 - Understock → demand exceeds inventory
 - Overstock → inventory exceeds demand
 - Balanced → inventory equals demand
 - Provides insights for inventory optimization

The third stage focuses on **automation using Jenkins CI/CD**, which transforms the system into a continuous and self-triggering pipeline. Instead of manual execution, the system is designed to:

- Detect dataset changes automatically
- Trigger the pipeline without human involvement
- Execute all stages in a predefined sequence
- Ensure consistency across runs

Another critical design aspect is the **integration of reporting mechanisms**, which ensures that processed data is converted into meaningful and actionable insights. This includes:

- Converting raw outputs into structured CSV files
- Generating visually structured PDF reports
- Highlighting key insights in an easy-to-understand format
- Automatically delivering reports via email

The overall system design is guided by the following principles:

- **Scalability**
 - Ability to handle increasing data sizes and additional nodes
- **Fault Tolerance**
 - Ensured through HDFS replication and distributed execution
- **Automation**
 - Eliminates manual effort using CI/CD pipelines
- **Modularity**
 - Independent components for easier maintenance and upgrades
- **Efficiency**
 - Parallel processing reduces execution time significantly

This methodology ensures that the system is not only efficient but also flexible and adaptable to future requirements such as real-time processing or integration with advanced analytics tools.

Implementation Steps

The implementation of this project follows a **step-by-step structured approach**, ensuring proper integration of all components and smooth execution of the complete pipeline. Each phase is carefully designed to build upon the previous one, resulting in a robust and fully automated system.

The process begins with the **setup of the Docker-based Hadoop cluster**. Containers are created for the NameNode and multiple DataNodes, and they are connected through a common Docker network. Configuration files are defined to enable HDFS and YARN services, ensuring proper communication and resource allocation across nodes.

Once the cluster is operational, the next step involves **data preparation and ingestion**. The dataset is initially stored in the local system and then:

- Copied into the NameNode container
- Uploaded into HDFS
- Automatically split into blocks
- Distributed across DataNodes with replication

The core processing logic is implemented using **Java-based MapReduce programs**, where two main jobs are developed:

- **SalesAnalysis Job**
 - Computes category-wise revenue
- **StockAnalysis Job**
 - Identifies inventory gaps and classifications

The implementation of these jobs involves:

- Writing Mapper and Reducer classes
- Compiling Java source files
- Packaging them into executable JAR files
- Submitting jobs to the Hadoop cluster
- Monitoring execution through YARN

After execution, the system generates output in multiple reducer partitions. These outputs are then:

- Retrieved from HDFS
- Merged into final consolidated files
- Verified for correctness and completeness

The next phase involves **Python-based automation**, which enhances usability and presentation of results. Scripts are developed to:

- Parse raw MapReduce outputs
- Convert them into structured CSV files
- Generate a well-formatted PDF report using ReportLab
- Include tables, summaries, and insights in the report
- Send the report via email using SMTP

To achieve complete automation, a **Jenkins CI/CD pipeline** is configured with multiple stages:

- **Cluster Initialization** — Start Docker containers and services
- **HDFS Preparation** — Upload dataset and clean previous outputs
- **MapReduce Execution** — Run distributed jobs
- **Report Generation** — Process outputs and create reports
- **Result Storage** — Copy outputs to local system
- **Cluster Shutdown** — Stop all services and containers

Finally, all outputs including text files, CSV files, and PDF reports are copied from the container environment to the local system, ensuring easy accessibility.

The implementation ensures that:

- The entire workflow runs automatically once triggered
- Manual intervention is completely eliminated
- Errors are handled through controlled pipeline execution
- The system remains stable and repeatable across multiple runs

Web UI Access Points

During demonstration and evaluation, several web-based interfaces are available to monitor and visually inspect different components of the system. These interfaces provide real-time insights into the working of the Hadoop cluster, job execution, and automation pipeline.

The following web interfaces are used:

URL	Purpose
http://localhost:9870	HDFS NameNode UI — Displays file system structure, block distribution, storage usage, and DataNode health status
http://localhost:8088	YARN ResourceManager UI — Shows running, completed, and failed MapReduce jobs along with resource allocation details
http://localhost:8080	Jenkins Dashboard — Displays CI/CD pipeline execution, build history, logs, and stage-wise progress

These interfaces are especially useful during demonstrations, as they allow:

- Verification of Hadoop cluster status
- Monitoring of data storage and replication in HDFS
- Tracking execution of MapReduce jobs in real time
- Observing Jenkins pipeline stages and automation flow

The availability of these dashboards improves system transparency and helps in debugging, validation, and performance analysis.

Technology Stack

The project integrates multiple technologies to build a complete end-to-end Big Data processing and automation pipeline. Each technology plays a specific role in ensuring scalability, efficiency, and reliability.

Core Big Data Technologies

- **Apache Hadoop (3.x)**
 - Main framework for distributed data processing
 - Executes MapReduce jobs across multiple nodes
- **HDFS (Hadoop Distributed File System)**
 - Stores large datasets in a distributed manner

- Splits data into blocks and replicates them across DataNodes
- Ensures fault tolerance and high availability
- **YARN (Yet Another Resource Negotiator)**
 - Manages cluster resources and job scheduling
 - Allocates CPU and memory for MapReduce tasks

Processing and Development

- **Java (MapReduce)**
 - Used to implement Mapper and Reducer programs
 - Handles distributed data processing logic
- **Python**
 - Used for post-processing and automation
 - Generates CSV files, PDF reports, and email notifications

Containerization

- **Docker**
 - Used to create a multi-node Hadoop cluster
 - Simulates distributed environment on a single machine
 - Ensures consistency and easy deployment

Automation and CI/CD

- **Jenkins**
 - Implements automated CI/CD pipeline
 - Monitors GitHub repository for changes
 - Executes full workflow from cluster startup to report generation
- **GitHub**
 - Stores project code and dataset
 - Acts as trigger source for Jenkins pipeline

Reporting and Output Generation

- **Python CSV Module**

- Converts raw MapReduce outputs into structured CSV format
- **ReportLab**
 - Generates formatted PDF reports
 - Includes tables and structured analysis

Communication

- **SMTP (Gmail SMTP Server)**
 - Sends generated reports via email
 - Ensures automatic delivery to users

Key Strengths of the Technology Stack

- **Distributed Processing** → Faster execution of large datasets
- **Fault Tolerance** → Data replication ensures reliability
- **Automation** → CI/CD eliminates manual effort
- **Scalability** → Easy to extend cluster with more nodes
- **Integration** → Smooth interaction between all components

Results and Discussion

The proposed system was successfully implemented and evaluated using a retail dataset containing approximately 73,000 records with multiple attributes such as product category, inventory levels, customer demand, pricing, and discounts. The execution of the project demonstrated the effective integration of distributed computing, automation, and reporting into a unified pipeline capable of handling large-scale data processing tasks.

The Hadoop-based distributed cluster played a crucial role in ensuring efficient data processing. Once the dataset was uploaded into HDFS, it was automatically divided into smaller blocks and distributed across multiple DataNodes. This enabled parallel execution of MapReduce jobs, significantly reducing the overall processing time compared to traditional single-node approaches. The replication mechanism ensured that data was reliably stored and protected against node failures.

The **Sales Analysis MapReduce job** produced category-wise revenue outputs by aggregating values across the dataset. The results clearly highlighted the contribution of different product categories to overall revenue. Categories with higher sales volume and pricing showed significantly larger revenue figures, providing valuable insights into business performance. The use of multiple reducers ensured that the workload was distributed efficiently, resulting in partitioned outputs that were later merged into a final consolidated file.

Similarly, the **Stock Analysis MapReduce job** generated detailed insights into inventory conditions at the product level. By comparing inventory levels with customer demand, the system was able to classify products into understock, overstock, and balanced categories. This classification is particularly useful in real-world scenarios where businesses need to maintain optimal stock levels to avoid losses due to overstocking or missed sales due to understocking.

The results of the analysis can be summarized as follows:

- **Accurate revenue computation** for each product category based on real transactional data
- **Clear identification of inventory gaps**, enabling better stock management decisions
- **Efficient parallel processing** using multiple reducers across DataNodes
- **Reliable data handling** through HDFS replication and distributed storage

After the MapReduce execution, the system generated multiple output partitions (part files), which were successfully merged into final output files. These outputs were then processed using Python scripts to generate structured CSV files, making the data easier to interpret and analyse.

The generation of the **PDF report** added significant value to the system by presenting the results in a professional and visually structured format. The report included:

- A summary of cluster configuration
- Category-wise revenue tables
- Product-wise stock analysis with classification
- Color-coded indicators for stock status:
 - Red for understock
 - Yellow for overstock
 - Green for balanced

This visual representation improves readability and allows stakeholders to quickly understand key insights without analysing raw data.

One of the most important aspects of the project is the **automation achieved through the Jenkins CI/CD pipeline**. The pipeline successfully detected dataset changes and executed the entire workflow automatically, including cluster setup, data processing, report generation, and email delivery. This eliminated the need for manual intervention and ensured consistency across multiple runs.

The automated workflow demonstrated the following key outcomes:

- End-to-end pipeline execution without manual steps
- Successful integration of Docker, Hadoop, and Jenkins
- Reliable execution of MapReduce jobs across multiple nodes
- Automatic generation and delivery of reports

From a performance perspective, the system showed strong scalability and efficiency. As the data was processed in parallel, the execution time remained manageable even with a relatively large dataset. The use of distributed architecture ensures that the system can handle even larger datasets by simply adding more nodes to the cluster.

However, during the implementation and testing phase, certain observations were made:

- Initial cluster startup requires some time for services to stabilize
- Proper configuration of HDFS and YARN is critical for smooth execution
- Data preprocessing is important to avoid inconsistencies in MapReduce output
- Network and container management must be handled carefully in Docker environments

Despite these minor challenges, the system performed reliably and achieved all intended objectives.

Overall, the results clearly demonstrate that the integration of **Hadoop MapReduce with CI/CD automation** provides a powerful solution for large-scale data processing. The system not only improves processing efficiency but also ensures automation, reliability, and ease of use. The generated insights are meaningful and can directly support decision-making in retail environments, making the solution both practical and scalable.

 **Jenkins** / BD_Project / Git Polling Log

 Changes

 Stages

 Rename

 Pipeline Syntax

 Git Polling Log

 Delete Pipeline

Git Polling Log

```
Started on 10-Apr-2026, 11:13:57 pm
Using strategy: Default
[poll] Last Built Revision: Revision a0aa343e3a673cc678b650d743ba2042f008ef13
(refs/remotes/origin/main)
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> C:\Program Files\Git\bin\git.exe --version # timeout=10
> git --version # 'git version 2.49.0.windows.1'
> C:\Program Files\Git\bin\git.exe ls-remote -h -- https://github.com/Dhaya09/BD_Project.git #
timeout=10
Found 1 remote heads on https://github.com/Dhaya09/BD_Project.git
[poll] Latest remote head revision on refs/heads/main is: a0aa343e3a673cc678b650d743ba2042f008ef13 -
already built by 22
Done. Took 1 sec
No changes
```



</> Changes

Stages

Rename

Pipeline Syntax

Git Polling Log

Delete Pipeline

Git Polling Log

```
Started on 10-Apr-2026, 11:17:57 pm
Using strategy: Default
[poll] Last Built Revision: Revision a0aa343e3a673cc678b650d743ba2042f008ef13
(refs/remotes/origin/main)
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> C:\Program Files\Git\bin\git.exe --version # timeout=10
> git --version # 'git version 2.49.0.windows.1'
> C:\Program Files\Git\bin\git.exe ls-remote -h -- https://github.com/Dhaya09/BD_Project.git #
timeout=10
Found 1 remote heads on https://github.com/Dhaya09/BD_Project.git
[poll] Latest remote head revision on refs/heads/main is: bdf97fccae95ca48388fe25a2472e338d2a0060b
Done. Took 1.1 sec
Changes found
```



Containers [Give feedback](#)

Container CPU usage

401.80% / 1800% (18 CPUs available)

Container memory usage

1005.8MB / 7.38GB

[Show charts](#)

Search



Only running

		Name	Container ID	Image	Port(s)	Actions
☐	☒	datanode3	e3320802530d	ubuntu:22.		
☐	☒	datanode2	4cca08691bad	ubuntu:22.		
☐	☒	datanode1	95579d0641a8	ubuntu:22.		
☐	☒	namenode	acb564615512	ubuntu:22.	8088:8088 Show all ports (2)	
☐	☐	hadoop-node	2496f4caa696	ubuntu:22.	8088:8088 Show all ports (2)	

Showing 5 items

The screenshot displays the Hadoop web interface for browsing the directory `/user/student/output/academic`. The interface features a green navigation bar at the top with tabs for Overview, Datanodes, Datanode Volume Failures, Snapshot, Startup Progress, and Utilities. Below the navigation bar, the main content area is titled "Browse Directory". It includes a search bar with the path `/user/student/output/academic` and a "Go!" button. To the right of the search bar are icons for file operations. Below the search bar, there is a "Show" dropdown set to "25" and a "Search:" input field. The main part of the interface is a table listing the contents of the directory. The table has columns for Permission, Owner, Group, Size, Last Modified, Replication, Block Size, and Name. The files listed are `_SUCCESS`, `part-r-00000`, `part-r-00001`, and `part-r-00002`, all owned by `root` and belonging to the `supergroup`. The table also shows the size of each file (0 B, 240 B, 160 B, and 239 B respectively) and the number of replicas (3 for each). At the bottom of the table, it says "Showing 1 to 4 of 4 entries" and provides navigation buttons for "Previous", "1", and "Next".

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	root	supergroup	0 B	Apr 09 12:26	3	128 MB	<code>_SUCCESS</code>
-rw-r--r--	root	supergroup	240 B	Apr 09 12:26	3	128 MB	<code>part-r-00000</code>
-rw-r--r--	root	supergroup	160 B	Apr 09 12:26	3	128 MB	<code>part-r-00001</code>
-rw-r--r--	root	supergroup	239 B	Apr 09 12:26	3	128 MB	<code>part-r-00002</code>



All Applications

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources	Total Resources
2	0	2	0	<memory:0 B, vCores:0>	<memory:24 GB, vCores:1>	

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes
3	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[memory-mb (unit=M), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	Average Yarn Utilization
application_1775843327323_0002	root	Stock Analysis	MAPREDUCE		default	0	Fri Apr 10 23:19:48 +0550 2026	Fri Apr 10 23:19:50 +0550 2026	Fri Apr 10 23:20:00 +0550 2026	FINISHED	SUCCEEDED	N/A	N/A
application_1775843327323_0001	root	Sales Analysis	MAPREDUCE		default	0	Fri Apr 10 23:19:19 +0550 2026	Fri Apr 10 23:19:20 +0550 2026	Fri Apr 10 23:19:43 +0550 2026	FINISHED	SUCCEEDED	N/A	N/A

Showing 1 to 2 of 2 entries

Jenkins / BD_Project / #23 / Stages

OverviewStagesConsoleChanges

#23

Started by an SCM changeStarted 3 min 12 sec agoQueued 8 secTook 2 min 57 sec

Graph

StartCheckout SCMStart ClusterPrepare HDFSRun MapReduceGenerate ReportsCopy Results to LocalStop ClusterPost ActionsEnd

Search

Checkout SCM 2s

Start Cluster 32s

Prepare HDFS 28s

Run MapReduce 54s

Generate Reports 28s

Copy Results to Local 3s

Stop Cluster 21s

Post Actions 1s

Post Actions1sStarted 16s agoJenkins

Windows Batch Scriptdocker exec namenode /opt/hadoop/sbin/stop-yarn.sh || exit 0 >0.31s

Windows Batch Scriptdocker exec namenode /opt/hadoop/sbin/stop-dfs.sh || exit 0 >0.33s

Windows Batch Scriptdocker stop namenode datanode1 datanode2 datanode3 || exit 0 >0.32s

```
0
1 C:\ProgramData\Jenkins\workspace\BD_Project>docker stop namenode datanode1 datanode2 datanode3 || exit 0
2 namenode
3 datanode1
4 datanode2
5 datanode3
```

OneDrive > OneDrive - Personal > Desktop > BD_Dataset > Results

SortView

Name	Status	Date modified	Type	Size
sales_analysis		4/10/2026 11:20 PM	Microsoft Excel Co...	1 KB
sales_final		4/10/2026 11:20 PM	Text Document	1 KB
sales_part0		4/10/2026 11:20 PM	Text Document	1 KB
sales_part1		4/10/2026 11:20 PM	Text Document	1 KB
sales_part2		4/10/2026 11:20 PM	Text Document	1 KB
sales_report		4/10/2026 11:20 PM	Microsoft Edge PD...	5 KB
stock_analysis		4/10/2026 11:20 PM	Microsoft Excel Co...	1 KB
stock_final		4/10/2026 11:20 PM	Text Document	1 KB
stock_part0		4/10/2026 11:20 PM	Text Document	1 KB
stock_part1		4/10/2026 11:20 PM	Text Document	1 KB
stock_part2		4/10/2026 11:20 PM	Text Document	1 KB

Retail Sales & Inventory Analysis Report

Generated: 2026-04-10 17:50:31

Apache Hadoop MapReduce | 3-Node Distributed Cluster on Docker

Distributed Cluster Configuration

Component	Details
NameNode	namenode — Master (HDFS + YARN ResourceManager)
DataNode 1	datanode1 — Worker (Storage + Map tasks)
DataNode 2	datanode2 — Worker (Storage + Map tasks)
DataNode 3	datanode3 — Worker (Storage + Map tasks)
Replication	3 — fault tolerance across nodes
Reducers	3 — parallel output partitions

Job 1: Revenue Analysis by Category

Category	Total Revenue
Electronics	97385585.94
Toys	98729216.66
Furniture	100230824.04
Groceries	99948968.32
Clothing	98676779.98

Job 2: Inventory Gap Analysis (Stock vs Demand)

Product ID	Avg Gap	Status
P0003	134.80	Overstock
P0006	132.56	Overstock
P0009	132.91	Overstock
P0012	132.95	Overstock

Gmail

Search mail

1 of 1,833

Retail Sales & Inventory Analysis Report — 2026-04-10

studentbubble00@gmail.com

Hello,

Please find attached the Retail Sales & Inventory Analysis Report generated using Apache Hadoop MapReduce on a 3-Node Distributed Cluster (Docker).

Cluster Setup:

- 1 NameNode (Master)
- 3 DataNodes (Workers)
- Data distributed across HDFS
- Parallel MapReduce execution with multiple reducers

Report includes:

- Job 1: Sales Analysis**
 - Total Revenue per Product Category
 - Business insights on top-performing categories
- Job 2: Stock Analysis**
 - Inventory vs Demand Gap per Product
 - Identification of Understock / Overstock items

This report helps in:

- Revenue optimization
- Inventory management
- Demand forecasting validation

- Demand forecasting validation

Regards,
Hadoop Retail Analytics Pipeline

One attachment • Scanned by Gmail • Add to Drive



Conclusion

This project successfully demonstrates the design and implementation of a **fully automated Big Data processing pipeline** for retail sales and inventory analysis using Apache Hadoop MapReduce in a Dockerized distributed environment. The system effectively integrates multiple technologies, including Hadoop, Docker, Jenkins, and Python, to create a seamless workflow that processes large datasets efficiently and delivers meaningful insights.

One of the key achievements of this project is the ability to **handle large-scale data in a distributed manner**. By leveraging HDFS for storage and MapReduce for parallel computation, the system overcomes the limitations of traditional centralized processing approaches. The use of multiple DataNodes ensures that data processing is performed concurrently, resulting in improved performance and reduced execution time.

Another major contribution is the implementation of a **fully automated CI/CD pipeline using Jenkins**, which eliminates the need for manual intervention. The system is capable of detecting dataset changes, triggering the pipeline, executing MapReduce jobs, generating reports, and delivering outputs automatically. This level of automation ensures consistency, reliability, and efficiency in repeated executions.

The analytical capabilities of the system provide **valuable business insights**, including:

- Identification of high-revenue product categories
- Detection of inventory imbalances such as understock and overstock
- Structured representation of results through CSV and PDF reports

The integration of reporting mechanisms further enhances the usability of the system by transforming raw outputs into clear and actionable information. The automated email delivery ensures that stakeholders receive timely updates without needing to access the system manually.

Overall, the project highlights the effectiveness of combining **distributed computing with DevOps practices** to build scalable and production-ready data processing systems. It not only meets the objectives defined at the beginning but also reflects real-world industry approaches to Big Data analytics and automation.

Future Work

While the current system achieves its intended goals, there are several opportunities for further enhancement and extension. Future improvements can focus on increasing system capabilities, improving performance, and expanding real-world applicability.

One potential direction is the integration of **real-time data processing frameworks**. Currently, the system operates in a batch-processing mode using MapReduce. Incorporating streaming technologies can enable continuous data processing and real-time analytics, which would be beneficial for dynamic retail environments.

Another important enhancement would be the development of an **interactive dashboard**. Instead of relying solely on PDF reports, a web-based dashboard could provide real-time visualization of

sales trends, inventory status, and system metrics. This would improve user interaction and make data analysis more intuitive.

The system can also be extended by incorporating **machine learning models** to provide predictive insights. For example:

- Forecasting future sales trends
- Predicting inventory demand
- Identifying potential stock shortages in advance

Additionally, the scalability of the system can be further improved by deploying it on **cloud platforms** such as AWS, Azure, or Google Cloud. This would allow dynamic resource allocation and enable the system to handle significantly larger datasets in production environments.

Other possible enhancements include:

- Improving fault tolerance with advanced monitoring tools
- Integrating alert systems for critical inventory conditions
- Enhancing data preprocessing for handling noisy or incomplete data
- Supporting multiple datasets and multi-source data integration

Finally, the system can be extended into a **full-fledged enterprise solution** by exposing its functionalities through REST APIs, enabling integration with external applications and business systems.

References

- [1] K. Keerthana *et al.*, "Big Data in Cloud Computing with HDFS and Spark – The Future of Analytics," *Proc. National Conf. on Cutting Edge Technologies*, Chettinad College of Engineering and Technology, India, 2017.
- [2] Z. Xia, A. Liu, *et al.*, "Contemporary Recommendation Systems on Big Data and Their Applications: A Survey," *arXiv preprint arXiv:2206.02631*, 2022.
- [3] J. Camacho-Rodríguez *et al.*, "Apache Hive: From MapReduce to Enterprise-Grade Big Data Warehousing," *arXiv preprint arXiv:1903.10970*, 2019.
- [4] Anonymous, "Case Study: Walmart's Inventory Optimization Using Hadoop," *Industry Case Study Report*, Scribd Publications, 2025.
- [5] IJRPR Authors, "E-Commerce Data Analysis Using Hadoop," *International Journal of Recent Progress in Research*, vol. 3, no. 4, 2023.
- [6] JISem Journal Authors, "Big Data-Based Recommendation Systems," *Journal of Interdisciplinary Systems and Management*, 2025.
- [7] M. El Yazidi *et al.*, "A Comparative Analysis of Hadoop and Spark Frameworks for Big Data Processing," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 4, 2021.
- [8] A. M. Al Jabri *et al.*, "Analysis on Sales Data Using Hadoop and MapReduce Algorithm," *Journal of Scientific Research*, 2023.

- [9] Atlantis Press Authors, "A Comprehensive Review on Big Data Recommendation Systems," *Atlantis Press Proceedings*, 2024.
- [10] C. D. T. Guerrero *et al.*, "Big Data Analytics: A Preliminary Study of Open Source Platforms," *Proc. 7th Int. Joint Conf. on Knowledge Discovery*, SciTePress, 2017.
- [11] R. Jeswani, "Predicting Walmart Sales: Exploratory Data Analysis and Visualization," *Capstone Project Report*, Rochester Institute of Technology, USA, 2021.
- [12] IJRAR Authors, "Recommendation System by Using Hadoop Framework," *International Journal of Research and Analytical Reviews*, vol. 7, no. 2, 2020.
- [13] R. R. Karthikeyan and B. Raghu, "Simple Analytics in Retail Sales Data Using Hadoop," *International Journal of Computer Trends and Technology*, vol. 58, no. 1, 2019.
- [14] H. Madhavan, "Vendor Performance Analysis Dataset," *Kaggle Dataset Repository*, 2023.
- [15] Ksolves Team, "Apache Hive vs. Apache Spark," *Ksolves Big Data Technical Blog*, 2026.
- [16] IJERT Authors, "Building Personalised Recommendation System with Big Data and Hadoop MapReduce," *International Journal of Engineering Research & Technology*, vol. 3, no. 4, 2014.
- [17] IEEE Authors, "Efficient Big Data Processing and Recommendation System Based on Apache Spark," *IEEE Conference Proceedings*, 2024.
- [18] SIC Authors, "A Context-Aware Recommendation System Based on Hadoop and MapReduce," *Studies in Informatics and Control*, 2025.
- [19] W. Wang *et al.*, "The Application of E-Commerce Recommendation System Based on Hadoop Platform," *COMSIS Journal*, 2020.
- [20] A. Sharma and S. Gupta, "BigData: A Case Study of Spark MLlib and Hive," *Technical Report*, 2018.
- [21] Y. Li *et al.*, "A Big Data Analysis Framework Using Apache Spark and Deep Learning," *arXiv preprint arXiv:1711.09279*, 2017.
- [22] S. Patil and S. Kulkarni, "Building Personalised Recommendation System with Big Data and Hadoop MapReduce," *IJERT Conference Paper*, 2014.
- [23] A. Popescu *et al.*, "A Context-Aware Recommendation System Based on Hadoop and MapReduce," *Studies in Informatics and Control*, 2025.
- [24] X. Chen, "Big Data in Online Retail Analysis," *Technical Report*, 2023.
- [25] S. Guleria and S. Sood, "Predictive Intelligence Using Big Data and IoT," *Research Collection*, 2020.
- [26] M. Vanani and A. Mohammadipour, "Optimizing Big Data Management and Industrial Systems," *Research Collection*, 2022.
- [27] J. Smith *et al.*, "Advancing Polyglot Big Data Processing Using the Hadoop Ecosystem," *arXiv preprint arXiv:2504.14322*, 2024.
- [28] Q. Zhang *et al.*, "Components and Development in Big Data System: A Survey," *Big Data Research Journal*, 2019.
- [29] IEEE Authors, "From Data to Decisions: Big Data Analytics in Action," *IEEE Conference Proceedings*, 2023.
- [30] R. Ladeira *et al.*, "Big Data Analytics and AI in Service Operations," *Service Industries Journal*, 2024.